Contents

Notes

The descriptions here are *very* brief. You will not be able to learn shell syntax from them; see the various references below. In particular the completion system is extremely rich and the descriptions of its utility functions are the barest memory joggers.

The start and end of each section is aligned with page boundaries, so you can print out only the parts you want to refer to.

References

Zsh manual: Supplied with the shell: should be installed in Unix manual page and info formats. Texinfo generates PS or PDF; available as separate doc bundle from same place as the shell.

http://zsh.sourceforge.net/: Site with much information about zsh, including HTML manual and a more user-friendly guide to the shell, as well as the FAQ.

Zsh wiki: http://www.zshwiki.org/: Extensible zsh web pages written by users.

From Bash to Z Shell: Conquering the Command Line, by Oliver Kiddle, Jerry Peek and Peter Stephenson, Apress, ISBN 1 59059 376 6. Introduction to interactive use of Unix shells in general in part 1, concentrating on bash and zsh in parts 2 and 3. The contents of the book are as follows; where noted with page references to this card they expand on the brief hints here.

Part 1 (Introducing the Shell) contains the following chapters:

Part 2 (Using bash and zsh) contains the following chapters:

Part3 (Extending the Shell) contains the following chapters:

The three appendices contain short descriptions of standard Unix programs, links to external resources, and a glossary.

Zsh manual pages

To access documentation from within the shell, use the **man** command with one of the following arguments:

zsh	Introduction, startup and shutdown
zshmisc	Syntax, redirection, functions, jobs, tests
zshexpn	Expansion and substitution
zshparam	Parameters (variables)
zshoptions	Options to the shell
zshbuiltins	Shell builtin commands
zshzle	The line editor, excluding completion
zshcompwid	The low-level completion facitilities
zshcompsys	The new completion system (more readable)
zshcompctl	The old completion system (deprecated)
zshmodules	Modules loadable with **zmodload**
zshtcpsys	Functions for using raw TCP via builtins
zshzftpsys	Functions for using FTP via builtins
zshcontrib	Contributed functions for zle etc.
zshall	Everything in one large manual page

Mailing lists

zsh-users@zsh.org: users' mailing list for general questions and tips; to join, mail **zsh-users-subscribe@zsh.org**.

zsh-workers@zsh.org: mailing list for bug reports, patches and developers' discussions; to join, mail **zsh-workers-subscribe@zsh.org**. New developers with some Unix/Linux experience are welcome.

Grammar

List is any sequence of *sublists* (including just one) separated by **;** or **newline**. **;** and **newline** are always interchangeable except in **;;**.

Sublist is any sequence of *pipelines* (including just one) connected by **&&** or **||**.

Pipeline is any sequence of simple commands connected by **|**.

Command is either a simple command (a command *word)* followed optionally by *word* … or one of the special commands below.

Word is any text that produces a single word when expanded; *word* … is any number of these separated by whitespace.

Name is a shell identifier: an alphabetic character or **_** followed by any sequence of alphanumeric characters or **_**.

[…] indicates optional; dots on their own line mean any number of repetitions of the line just above.

Bold text is to be typed literally.

Status "true" or "false" is determined by: for commands, the return status; for pipelines the last command; for sublists the last pipeline; for lists the last sublist that was executed.

sublist1 **&&** *sublist2* [**&&** *sublist3* …]
Execute *sublists* until one is false.

sublist1 **||** *sublist2* [**||** *sublist2* …]
Execute *sublists* until one is true. Note strings of **&&** sublists can contain **||** sublists and vice versa; they are parsed left to right.

command1 **|** command2 [**|** command3 …]
Execute *command1*, sending its output to the input of *command2*, and so on (a *pipeline*).

if *listi1*; **then**[;] *listt1;*
[**elif** *listi2;* **then** *listt2;*]
…
[**else** *listt3;*]
fi
If *listi*1 is true, execute *listt1*; else if *listi2* is true execute *listt2*; else execute *listt3*.

for *name* [**in** *word* …]
do *list*;
done
Execute *list* with variable *name* set to each of *word* … in turn If **in …** is omitted the positional parameters are used.

for *name* **in** *word* …; **{** *list* **}**
foreach *name* **(** *word* … **)** [;]
list;
end
Non-portable alternative forms.

while *listw*; **do** *listd*; **done**
While *listw* is true execute *listd*.

until *listu*; **do** *listd*; **done**
Non-portable: while *listu* is not true execute *listd*.

repeat *numexp*; **do** *list*; **done**
repeat *numexp sublist*
Non-portable: repeat *list* or *sublist numexp* times.

case *word* **in**
[(] *pattern1*[|*pattern2...*]**)** [;] *list* **;;**
…
esac
Try matching word against every pattern in turn until success. Execute the corresponding list. **;&** instead of **&&** means fall through to next *list*.

case *word* **{**
[(] *pattern1*[|*pattern2...*]**)** [;] *list* **;;**
…
}
Non-portable alternative.

select *name* [**in** *word …*]**;**
do *list*;
done
Print menu of *words*, read a number, set *name* to selected word, execute *list* until end of input. Portable but rare.

(*list*[;]**)**
Execute *list* in a subshell (a new process where nothing that happens affects the current shell).

{*list*[;]**}**
Execute *list* (no new process: simply separates list from what's around and can take redirections).

function *nameword* **{**[;] *list*[;] **}**
nameword **() {[;]** *list*[;] **}**
Define function named *nameword*; executes list when run; running *nameword word1* … makes *word1* … available as **$1** etc. in function body. *list* must end with [;] or newline for portability. *nameword* can be repeated to define multiple functions (rare, non-portable).

time [*pipeline*]
Report time for *pipeline* if given else totals for current shell.

[[*condition* **]]**
Evaluate *condition* (see below), gives status true or false.

Pattern matching (globbing)

Basic patterns:

`*`	Any string
`?`	Any character
`[`*class*`]`	Any single character from *class*
`[^`*class*`]`	Any single character not from class
`<`*num1*`-`*num2*`>`	Any number between *num1* and *num2*
	`<-`*num2*`>` from 0; `<`*num1*`->` to infinity.
`**/`	Directories to any level
`(`*pat1*`)`	Group patterns
`(`*pat1*`|`*pat2*`)`	*pat1* or *pat2* (any number of `|`'s)

Character classes may contain any character or the following special patterns in any mix; literal – must be first; literal ^ must not be first:

a`-`*b*	A character in the range *a* to *b*
`[:alnum:]`	An alphanumeric character
`[:alpha:]`	An alphabetic character
`[:ascii:]`	A character in the ASCII character set
`[:blank:]`	A space or tab
`[:cntrl:]`	A control character
`[:digit:]`	A decimal digit
`[:graph:]`	A printable character other than whitespace
`[:lower:]`	A lower case letter
`[:print:]`	A printable character
`[:punct:]`	A punctuation character
`[:space:]`	Any whitespace character
`[:upper:]`	An upper case letter
`[:xdigit:]`	A hexadecimal digit

Extended patterns (option **EXTENDED_GLOB** must be set):

`^`*pat*	Anything that doesn't match *pat*
pat1`^`*pat2*	Match *pat1* then anything other than *pat2*
pat1`~`*pat2*	Anything matching *pat1* but not *pat2*
X`#`	Zero or more occurrences of element *X*
X`##`	One or more occurrences of element *X*

KSH_GLOB operators (patterns may contain `|` for alternatives):

`@(`*pat*`)`	Group patterns
`*(`*pat*`)`	Zero or more occurrences of *pat*
`+(`*pat*`)`	One or more occurrences of *pat*
`?(`*pat*`)`	Zero or one occurrences of *pat*
`!(`*pat*`)`	Anything but the pattern *pat*

Globbing flags with **EXTENDED_GLOB**:

`(#i)`	Match case insensitively
`(#l)`	Lower case matches upper case
`(#I)`	Match case sensitively
`(#b)`	Parentheses set **match**, **mbegin**, **mend**
`(#B)`	Parentheses no longer set arrays
`(#m)`	Match in **MATCH**, **MBEGIN**, **MEND**
`(#M)`	Don't use **MATCH** etc.
`(#a`*num*`)`	Match with *num* approximations
`(#s)`	Match only at start of test string
`(#e)`	Match only at end of test string
`(#q`*expr*`)`	*expr* is a a set of glob qualifiers (below)

Glob qualifiers (in parentheses after file name pattern):

`/`	Directory
`F`	Non-empty directory; for empty use `(/^F)`
`.`	Plain file
`@`	Symbolic link
`=`	Socket
`p`	Name pipe (FIFO)
`*`	Executable plain file
`%`	Special file
`%b`	Block special file
`%c`	Character special file
`r`	Readable by owner (N.B. not current user)
`w`	Writeable by owner
`x`	Executable by owner
`A`	Readable by members of file's group
`I`	Writeable by members of file's group
`E`	Executable by members of file's group
`R`	World readable
`W`	World writeable
`X`	World executable
`s`	Setuid
`S`	Setgid
`t`	Sticky bit
`f`*spec*	Has **chmod**-style permissions *spec*
`e`*string*	Evaluation *string* returns true status
`+`*cmd*	Same but *cmd* must be alphanumeric or `_`
`d`*dev*	Device number *dev* (major*256 + minor)
`l[-+]`*num*	Link count is (less than, greater than) *num*
`U`	Owned by current effective UID
`G`	Owned by current effective GID
`u`*uid*	Owned by given *uid* (may be `<`*name*`>`)
`g`*gid*	Owned by given *gid* (may be `<`*name*`>`)
`a[Mwhms][-+]n`	Access time in given units (see below)
`m[Mwhms][-+]n`	Modification time in given units
`c[Mwhms][-+]n`	Inode change time in given units
`L[kmp][-+]n`	Size in given units (see below)
`^`	Negate following qualifiers
`-`	Toggle following links (first one turns on)
`M`	Mark directories
`T`	Mark directories, links, special files
`N`	Whole pattern expands to empty if no match
`D`	Leading dots may be matched
`n`	Sort numbers numerically
`o[nLlamcd]`	Order by given code (as below; may repeat)
`O[nLlamcd]`	Order by reverse of given code
`[`*num*`]`	Select *num*th file in current order
`[`*num1*`,`*num2*`]`	Select *num1*th to *num2*th file (as arrays)
`:`*X*	History modifier *X*; may have more

Time units are Month, week, hour, minute, second; default is day.
Size units are kilobytes, megabytes or 512-byte blocks (**p**); default is bytes; upper case means the same as lower case.
Order codes are name (default), size, link count, access time, modification time, inode change time, directory depth.

Expansion

Basic forms of expansion in the order they order:

`!`*expr*	History expansion
alias	Alias expansion
`<(`*cmds*`)`	Replaced by file with output from *cmds*
`=(`*cmds*`)`	Same but can be reread (use for **diff**)
`>(`*cmds*`)`	Replaced by file with input to *cmds*
`$`*var*	Variable substitution
`${`*var*`}`	Same but protected, allows more options
`$(`*cmds*`)`	Replaced by output of *cmds*
`` ``*cmds*`` `	Older form of same, harder to nest
`$((`*expr*`))`	Arithmetic result of evaluating *expr*
X`{a,b}`*Y*	*XaY Xby* (N.B. does no pattern matching)
X`{1..3}`*Y*	*X*`1`*Y X*`2`*y X*`3`*y*
X`{08..10}`*Y*	*X*`08`*Y X*`09`*y X*`10`*y*
`~`*user*, `~`*dir*	User home, named dir (*dir* is var name)
`=`*cmd*	/full/path/to/cmd
pattern	Glob file names, as above

History expansion:

`!!`	Immediately preceding line (all of it)
`!{!}`	Same but protected, may have args in {}
`!`	Line just referred to, default !!
`!13`	Line numbered 13 (**history** shows nos.)
`!-2`	Command two before current
`!`*cmd*	Last command beginning *cmd*
`!?`*str*	Last command containing *str*
`!#`	Current command line so far

Word selectors:

`!!:0`	Extract argument 0 (command word)
`!!:1`	Argument numbered 1 (first cmd arg)
`!!:^`	Also argument 1
`!!:$`	Last command argument
`!:%`	Word found by `!?str` (needs correct line)
`!!:2-4`	Word 2 to 4 inclusive
`!!:-4`	Words 0 to 4 inclusive
`!!:*`	Words 1 to $ inclusive
`!!:2*`	Words 2 to $ inclusive
`!!:2-`	Words 2 to $-1 inclusive

Modifiers on arguments (can omit word selector):

`!!:1:h`	Trailing path component removed
`!!:1:t`	Only trailing path component left
`!!:1:r`	File extension .ext removed
`!!:1:e`	Only extension ext left
`!!:1:p`	Print result but don't execute
`!!:1:q`	Quote from further substitution
`!!:1:Q`	Strip one level of quotes
`!!:1:x`	Quote and also break at whitespace
`!!:1:l`	Convert to all lower case
`!!:1:u`	Convert to all upper case
`!!:1:s/`*s1*`/`*s2*`/`	Replace string *s1* by *s2*
`!!:1:gs/`*s2*`/`*s2*`/`	Same but global
`!!:1:&`	Use same *s1* and *s2* on new target

Most modifiers work on variables (e.g **${var:h}**) or in glob qualifiers (e.g. ***(:h)**), the following only work there:

`${var:f`*m*`}`	Repeat modifier *m* till stops changing
`${var:F:`*N*`:`*m*`}`	Same but no more than *N* times
`${var:w`*m*`}`	Apply modifer *m* to words of string
`${var:W:`*sep*`:`*m*`}`	Same but words are separated by *sep*

Prompt expansion (with **PROMPT_PERCENT**, on by default); may take a decimal number *n* (default 0) immediately after the **%**:

%! %h	Current history event number
%#	**#** if superuser, else **%**
%%	A single **%**
%)	A **)** (use with **%**_X_**(.**_tstr_**.**_fstr_**)**)
%*	Time in 24-hour format with seconds
%/ %d	**$PWD**; _n_ gives trailing parts, **-**_n_ leading
%c %. %C	Deprecated alternatives, differ by default _n_
%?	Return status of last command
%@ %t	Time of day in am/pm format
%B (**%b**)	Start (stop) bold face mode
%D %D{_str_**}**	Date as *YY-MM-DD*, optional strftime spec
%E	Clear to end of line
%i	Script/function line number (**$LINENO**)
%j	Number of jobs as listed by jobs
%L	Shell depth (**$SHLVL**)
%l	Login terminal without **/dev** or **/dev/tty**
%M	Full host name
%m	Host name to first dot or _n_ dots
%N	Name of script, function, sourced file
%n	Name of user (same as **$USERNAME**)
%S %s	Start (stop) standout mode
%T	Time of day, 24-hour format
%U %u	Start (stop) underline mode (patchy support)
%v	_n_th component of **$psvar** array
%W	Date as middle-endian *MM/DD/YY*
%w	Date as *DAY DD*
%y	Login terminal without **/dev**
%_	Parser state (continuation lines, debug)
%~	Like **%/**, **%d** but with tilde substitution
%{_esc_**%}**	Escape sequence *esc* doesn't move cursor
%_X_**(.**_tstr_**.**_fstr_**)**	*tstr* if test _X_ gives _n_, else *fstr*
%<_str_**<**	Truncate to n on left, *str* on left if so
%>_str_**>**	Truncate to n on right, *str* on right if so

Test characters in **%**_X_**(.**_tstr_**.**_fstr_**)**: **!** Privileged; **#** uid _n_; **?** last status _n_; **_** at least _n_ nested constructs; **/** at least _n_ **$PWD** elements; **~** same with **~** subst; **D** month is _n_; **d** day of month is _n_; **g** effective gid is _n_; **j** at least _n_ jobs; **L** **$SHLVL** at least _n_; **l** at least _n_ chars on line so far; **S** **$SECONDS** at least _n_; **T** hours is _n_; **t** minutes is _n_; **v** at least _n_ components in **$psvar**; **w** day of week is _n_ (Sunday = 0).

Parameter (Variable) Expansion

Basic forms: *str* will also be expanded; most forms work on words of array separately:

${*var***}**	Substitute contents of *var*, no splitting
${+*var***}**	1 if *var* is set, else 0
${*var***:-***str***}**	**$***var* if non-null, else *str*
${*var***-***str***}**	**$***var* if set (even if null) else *str*
${*var***:=***str***}**	**$***var* if non-null, else *str* and set *var* to it
${*var***::=***str***}**	Same but always use *str*
${*var***:?***str***}**	**$***var* if non-null else error, abort
${*var***:+***str***}**	*str* if **$***var* is non-null
${*var***#***pat***}**	min match of *pat* removed from head
${*var***##***pat***}**	max match of *pat* removed from head
${*var***%***pat***}**	min match of *pat* removed from tail
${*var***%%***pat***}**	max match of *pat* removed from tail
${*var***:#***pat***}**	**$***var* unless *pat* matches, then empty
${*var***/***p***/***r***}**	One occurrence of *p* replaced by *r*
${*var***//***p***/***r***}**	All occurrences of *p* replaced by *r*
${#*var***}**	Length of *var* in words (array) or bytes
${^*var***}**	Expand elements like brace expansion
${=*var***}**	Split words of result like lesser shells
${~*var***}**	Allow globbing, file expansion on result
${${*var***%***p***}#***q***}**	Apply **%***p* then **#***q* to **$***var*

Parameter flags in parentheses, immediately after left brace:

%	Expand **%**s in result as in prompts
@	Array expand even in double quotes
A	Create array parameter with **${...=...}**
a	Array index order, so **Oa** is reversed
c	Count characters for **${#***var***}**
C	Capitalize result
e	Do parameter, comand, arith expansion
f	Split result to array on newlines
F	Join arrays with newlines between elements
i	**oi** or **Oi** sort case independently
k	For associative array, result is keys
L	Lower case result
n	**on** or **On** sort numerically
o	Sort into ascending order
O	Sort into descending order
P	Interpret result as parameter name, get value
q	Quote result with backslashes
qq	Quote result with single quotes
qqq	Quote result with double quotes
qqqq	Quote result with **$'...'**
Q	Strip quotes from result
t	Output type of variable (see below)
u	Unique: remove duplicates after first
U	Upper case result
v	Include value in result; may have **(kv)**
V	Visible representation of special chars
w	Count words with **${#***var***}**
W	Same but empty words count
X	Report parsing errors (normally ignored)
z	Split to words using shell grammar
p	Following forms recognize print \-escapes
j:str:	Join words with *str* between
l:x:	Pad with spaces on left to width *x*
l:x::s1:	Same but pad with repeated *s1*
l:x::s1::s2:	Same but *s2* used once before any *s1*s
r:x::s1::s2:	Pad on right, otherwise same as **l** forms
s:str:	Split to array on occurrences of *str*
S	With patterns, search substrings
I:exp:	With patterns, match *exp*th occurrence
B	With patterns, include match beginning
E	With patterns, include match end
M	With patterns, include matched portion
N	With patterns, include length of match
R	With patterns, include unmatched part (rest)

Delimeters shown as **:***str***:** may be any pair of chars or matched parenthses **(***str***)**, **{***str***}**, **[***str***]**, **<***str***>**.

Order of rules:

1. Nested substitution: from inside out
2. Subscripts: **${***arr***[3]}** extract word; **${***str***[2]}** extract character; **${***arr***[2,4]}**, **${***str***[4,8]}** extract range; **-1** is last word/char, **-2** previous etc.
3. **${(P)***var***}** replaces name with value
4. **¨$***array*¨ joins array, may use **(j:***str***:)**
5. Nested subscript e.g. **${${***var***[2,4]}[1]}**
6. **#**, **%**, **/** etc. modifications
7. Join if not joined and **(j:***str***:)**, **(F)**
8. Split if **(s)**, **(z)**, **(z)**, **=**
9. Split if **SH_WORD_SPLIT**
10. Apply **(u)**
11. Apply **(o)**, **(O)**
12. Apply **(e)**
13. Apply **(l.***str***.)**, **(r.***str***.)**
14. If single word needed for context, join with **$IFS[1]**.

Types shown with **(t)** have basic type **scalar**, **array**, **integer**, **float**, **assocation**, then hyphen-separated words from following list:

local	Parameter is local to function
left	Left justified with **typeset -L**
right_blanks	Right justified with **typeset -R**
right_zeros	Right justified with **typeset -Z**
lower	Lower case forced with **typeset -l**
upper	Upper case forced with **typeset -u**
readonly	Read-only, **typeset -r** or **readonly**
tag	Tagged as **typeset -t** (no special effect)
export	Exported with **export**, **typeset -x**
unique	Elements unique with **typeset -U**
hide	Variable not special in func (**typeset -h**)
hideval	**typeset** hides value (**typeset -H**)
special	Variable special to shell

Parameters (Variables)

Parameters set by shell, † denotes special to shell (may not be reused except by hiding with typeset -h in functions)

†!	Process ID of last background process
†#	Number of arguments to script or function
†ARGC	Same
†$	Process ID of main shell process
†-	String of single letter options set
†*	Positional parameters
†argv	Same
†@	Same, but does splitting in double quotes
†?	Status of last command
†0	Name of shell, usually reflects functions
†_	Last argument of previous command
CPUTYPE	Machine type (run time)
†EGID	Effective GID (via system call), set if root
†EUID	Effective UID (via system call), set if root
†ERRNO	Last system error number
†GID	Real group ID (via system call), set if root
HISTCMD	The current history line number
HOST	The host name
†LINENO	Line number in shell, function
LOGNAME	Login name (exported by default)
MACHTYPE	Machine type (compile time)
OLDPWD	Previous directory
†OPTARG	Argument for option handled by getopts
†OPTIND	Index of positional parameter in getopts
OSTYPE	Operating system type (compile time)
†pipestatus	Array giving statuses of last pipeline
†PPID	Process ID of parent of main shell
PWD	Current directory
†RANDOM	A pseudo-random number, repeating
†SECONDS	Seconds since shell started
†SHLVL	Depth of current shell
signals	Array giving names of signals
†status	Status of last command
†TRY_BLOCK_ERROR	In always block, 1 if error in try block
TTY	Terminal associated with shell if any
†TTYIDLE	Time for which terminal has been idle
†UID	Real user ID (via system call), set if root
†USERNAME	Name for $UID, set if root
VENDOR	Operating system vendor (compile time)
ZSH_NAME	Base name of command used to start shell
ZSH_VERSION	Version number of shell

Parameters used by the shell if set: **:** indicates arrays with corresponding colon-separated paths e.g. **cdpath** and **CDPATH**:

ARGV0	Export to set name of external command
BAUD	Baud rate: compensation for slow terminals
†cdpath :	Directories searched for **cd** target
†COLUMNS	Width of screen
DIRSTACKSIZE	Maximum size of stack for **pushd**
ENV	File to source when started as **sh** or **ksh**
FCEDIT	Default editor used by **fc**
†fignore :	List of suffixes ignored in file completion
†fpath :	Directories to search for autoloading
†histchars	History, quick replace, comment chars
†HISTCHARS	Same, deprecated
HISTFILE	File for reading and writing shell history
†HISTSIZE	Number of history lines kept internally
†HOME	Home directory for **~** and default **cd** target
†IFS	Characters that separate fields in words
KEYTIMEOUT	Time to wait for rest of key seq (1/100 s)
†LANG	Locale (usual variable, **LC_*** override)
†LC_ALL	Locale (overrides **LANG**, **LC_*)**
†LC_COLLATE	Locale for sorting etc.
†LC_CTYPE	Locale for character handling
†LC_MESSAGES	Locale for messages
†LC_NUMERIC	Locale for decimal point, thousands
†LC_TIME	Locale for date and time
†LINES	Height of screen
LISTMAX	Number of completions shown w/o asking
LOGCHECK	Interval for checking **$watch**
MAIL	Mail file to check (**$mailpath** overrides)
MAILCHECK	Mail check interval, secs (before prompt)
†mailpath :	List of files to check for new mail
†manpath :	Directories to find manual, used by **man**
†module_path :	Directories for **zmodload** to find modules
†NULLCMD	Command used if only redirection given
†path :	Command search path
†POSTEDIT	Termcap strings sent to terminal after edit
†PS1, PROMPT, prompt	Printed at start of first line of output; see above for escape sequences for all **PS**s
†PS2, PROMPT2	Printed for continuation lines
†PS3, PROMPT3	Print within **select** loop
†PS4, PROMPT4	For tracing execution (**xtrace** option)
†psvar :	Used with **%*n*v** in prompts
†READNULLCMD	Command used when only input redir given
REPORTTIME	Show report if command takes this long (s)
REPLY	Used to return a value e.g. by **read**
reply	Used to return array value
†RPS1, RPROMPT	Printed on right of screen for first line
†RPS2, RPROMPT2	Printed on right of screeen for continuation line
SAVEHIST	Max number of history lines saved
†SPROMPT	Prompt when correcting spelling
STTY	Export with **stty** arguments to command
†TERM	Type of terminal in use (**xterm** etc.)
TIMEFMT	Format for reporting usage with **time**
TMOUT	Send **SIGALRM** after seconds of inactivity
TMPPREFIX	Path prefix for shell's temporary files
†watch :	List of users or **all**, **notme** to watch for
WATCHFMT	Format of reports for **$watch**
†WORDCHARS	Chars considered parts of word by zle
ZBEEP	String to replace beeps in line editor
ZDOTDIR	Used for startup files instead of **~** if set

Tests and numeric expressions

Usually used after if, while, until or with && or ||, but the status may be useful anywhere e.g. as implicit return status for function.

File tests, e.g. **[[-e *file*]]**:

-a	True if *file* exists
-b	True if *file* is block special
-c	True if *file* is character special
-d	True if *file* is directory
-e	True if *file* exists
-f	True if *file* is a regular file (not special or directory
-g	True if *file* has setgid bit set (mode includes 02000)
-h	True if *file* is symbolic link
-k	True if *file* has sticky bit set (mode includes 02000)
-p	True if *file* is named pipe (FIFO)
-r	True if *file* is readable by current process
-s	True if *file* has non-zero size
-u	True if *file* has setuid bit set (mode includes 04000)
-w	True if *file* is writeable by current process
-x	True if *file* executable by current process
-L	True if *file* is symbolic link
-O	True if *file* owned by effective UID of current process
-G	True if *file* has effective GID of current process
-S	True if *file* is a socket (special communication file)
-N	True if *file* has access time no newer than mod time

Other single argument tests, e.g. **[[-n *str*]]**:

-n	True if *str* has non-zero length
-o	True if option *str* is set
-t	True if *str* (number) is open file descriptor
-z	True if *str* has zero length

Multiple argument tests e.g. **[[*a* -eq *b*]]**: numerical expressions may be quoted formulae e.g. **'1*2'**:

-nt	True if file *a* is newer than file *b*
-ot	True if file *a* is older than file *b*
-ef	True if *a* and *b* refer to same file (i.e. are linked)
=	True if string *a* matches pattern *b*
==	Same but more modern (and still not often used)
!=	True if string *a* does not match pattern *b*
<	True if string *a* sorts before string *b*
>	True if string *a* sorts after string *b*
-eq	True if numerical expressions *a* and *b* are equal
-ne	True if numerical expressions *a* and *b* are not equal
-lt	True if $a < b$ numerically
-gt	True if $a > b$ numerically
-le	True if $a \le b$ numerically
-ge	True if $a \ge b$ numerically

Combining expressions: *expr* is any of the above, or the result of any combination of the following:

(*expr*)	Group tests
! *expr*	True if *expr* is false and vice versa
exprA* && *exprB	True if both expressions true
exprA* || *exprB	True if either expression true

For complicated numeric tests use **((*expr*))** where *expr* is a numeric expression: status is 1 if *expr* is non-zero else 0. Same syntax used in **$((*expr*))** substitution. Precedences of operators from highest to lowest are:

- *func*(*arg...*), numeric constant (e.g. **3**, **-4**, **3.24**, **-14.6e-10**), *var* (does not require **$** in front unless some substitution e.g. **${#*var*}** is needed, **$** is error if *var* is to be modified)
- **(*expr*)**
- **!**, **~**, **++** (post- or preincrement), **--** (post- or predecrement), unary **+**, unary **-**
- **&**
- **^**
- **|**
- ****** (exponentiation)
- *****, **/**, **%**
- binary **+**, binary **-**
- **<<**, **>>**
- **<**, **<=**, **>**, **>=**
- **==**, **!=**
- **&&**
- **||**, **^^**
- **?** (ternary operator)
- **:** (true/false separator for ternary operator)
- **=**, **+=**, **-=**, ***=**, **/=**, **%=**, ****=**, **&=**, **^=**, **|=**, **<<=**, **>>=**, **&&=**, **^^=**, **||=**
- **,** (as in C, evaluate both sides and return right hand side).

For functions use **zmodload -i zsh/mathfunc**; functions available are as described in C math library manual:

- Single floating point argument, return floating point: **acos**, **acosh**, **asin**, **asinh**, **atan** (optional second argument like C atan2), **atanh**, **cbrt**, **ceil**, **cos**, **cosh**, **erf**, **erfc**, **exp**, **expm1**, **fabs**, **floor**, **gamma**, **j0**, **j1**, **lgamma**, **log**, **log10**, **log1p**, **logb**, **sin**, **sinh**, **sqrt**, **tan**, **tanh**, **y0**, **y1**
- Single floating point argument, return integer: **ilogb**
- No arguments, return integer: **signgam** (remember parentheses)
- Two floating point arguments, return floating point: **copysign**, **fmod**, **hypot**, **nextafter**
- One integer, one floating point argument, return floating point: **jn**, **yn**
- One floating point, one integer argument, return floating point: **ldexp**, **scalb**
- Either integer or floating point, return same type: **abs**
- Coerce to floating point: **float**
- Coerce to integer: **int**
- Optional string argument (read/write variable name), return floating point: **rand48**

Example use:

```
zmodload -i zsh/mathfunc
float x
(( x = 26.4 * sqrt(2) ))
print $(( log(x)/2 ))
```